Roll no: U24CS076

Name : Rushang Bagada

1. Write a C++ program to access public members of a class, by using the (.)dot operator.

These are members marked with a public access modifier.

Example: Price of Car A1 = 10000

Price of Bike A2 = 2000

Number of Car available in showroom = 4

Number of Bike available in showroom = 3

```
#include<bits/stdc++.h>
using namespace std;



class car{
public:
   int price;
   int number;

   car(int p, int n) : price(p), number(n) {}

   void display() {
      cout << "Price of Car  = " << price << endl;
      cout << "Number of Car available in showroom = " << number << endl;
   }
};

class bike{
```

```
public:
   int price;
   int number;

   bike(int p, int n) : price(p), number(n) {}

   void display() {
      cout << "Price of Bike  = " << price << endl;
      cout << "Number of Bike available in showroom = " << number << endl;
   }

};

int main(){
   car A1(10000, 4);
   bike A2(2000, 3);
   A1.display();
   A2.display();
   return 0;
}
```

```
Price of Car  = 10000
Number of Car available in showroom = 4
Price of Bike  = 2000
Number of Bike available in showroom = 3
```

2. Write a C++ program to create an object of a class and access class attributes

Example: Student's Roll No.: 101

Student's Name: First_Name Last_Name

Student's Percentage: 90.4

```
#include<bits/stdc++.h>
using namespace std;

class student{
public:
   int rollno;
   string name;
   float percentage;
   student(int r, string n, float p){
      rollno = r;
      name = n;
      percentage = p;
   }

   void display() {
      cout << "Student's Roll No.: " << rollno << endl;
      cout << "Student's Name: " << name << endl;
      cout << "Student's Percentage: " << percentage << endl;
   }
};

int main(){
   student s1(76,"rushang", 90.45);
   s1.display();}
```

```
Student's Roll No.: 76
Student's Name: rushang
Student's Percentage: 90.45
```

3. Write a C++ program to calculate the area of a circle using an area function where radius is a

private and area function is a public member of a class circle.

```
#include<bits/stdc++.h>

using namespace std;


class circle{

private:

  float radius;

public:

  circle(float r){

    radius = r;

  }

  float area(){

    return 3.14*radius*radius;

  }

};


int main(){

  circle c(5.0);

  cout << "Area of the circle with radius " << 5.0 << " is " << c.area() << endl;

  return 0;

}
```

```
Area of the circle with radius 5 is 78.5
```

4. Develop a Library class that contains a collection of books (Book objects). Implement methods

to add a book, remove a book, and display all books.

```
#include <bits/stdc++.h>


using namespace std;

class Book {

public:

   string title;

   string author;

   int year;

   Book(string t, string a, int y){

      title = t;

      author = a;

      year = y;

   }

   void display() {

      cout << "Title: " << title << ", Author: " << author << ", Year: " << year << endl;

   }

};


class Library{

   private:

       vector<Book> books;


   public:

    void addBook(string title,string author ,int year){

      Book b(title, author, year);
```

```
        books.push_back(b);
    }
    void removeBook(string title){
     for (auto it = books.begin();it !=books.end();it++){
        if (it->title == title) {
            books.erase(it);
            break;
        }
     }
    }
    void displayBooks() {
       for (int i = 0; i < books.size(); i++) {
          books[i].display();
       }
    }
};

int main (){
   Library lib;
   lib.addBook("The Great Gatsby", "F. Scott Fitzgerald", 1925);
   lib.addBook("1984", "George Orwell", 1949);
   lib.addBook("To Kill a Mockingbird", "Harper Lee", 1960);

   cout << "Books in the library:" << endl;
   lib.displayBooks();

   cout << "\nRemoving '1984' from the library." << endl;
   lib.removeBook("1984");
```

```
    cout << "Books in the library after removal:" << endl;
    lib.displayBooks();

    return 0;
}
```

```
Title: The Great Gatsby, Author: F. Scott Fitzgerald, Year: 1925
Title: 1984, Author: George Orwell, Year: 1949
Title: To Kill a Mockingbird, Author: Harper Lee, Year: 1960

Removing '1984' from the library.
Books in the library after removal:
Title: The Great Gatsby, Author: F. Scott Fitzgerald, Year: 1925
Title: To Kill a Mockingbird, Author: Harper Lee, Year: 1960
```

5. Create a BankAccount class with attributes accountNumber, accountHolder, and balance.

Implement methods to deposit and withdraw money. Ensure to display an appropriate message

if a withdrawal amount exceeds the available balance.

```
#include <bits/stdc++.h>

using namespace std;


class BankAccount {

private:

    string accountNumber;

    string accountHolder;

    double balance;


public:

    BankAccount(string number, string holder, double initialBalance) {
```

```
    accountNumber = number;
    accountHolder = holder;
    balance = initialBalance;
  }

  void deposit(double amount) {
    balance += amount;
    cout << "Deposit of $" << amount << " successful. New balance: $" << balance << endl;
  }

  void withdraw(double amount) {
    if (amount > balance) {
      cout << "Withdrawal of $" << amount << " failed. Insufficient balance." << endl;
    } else {
      balance -= amount;
      cout << "Withdrawal of $" << amount << " successful. New balance: $" << balance << endl;
    }
  }
};

int main() {
  BankAccount account("123456789", "rushang", 1000.0);

  account.deposit(500.0);
  account.withdraw(200.0);
  account.withdraw(1500.0);
```

```
    return 0;
}
```

```
Deposit of $500 successful. New balance: $1500
Withdrawal of $200 successful. New balance: $1300
Withdrawal of $1500 failed. Insufficient balance.
```

6. Create a C++ program that demonstrates function overloading using a class. You are required

to design a class named Calculator that performs addition operations. The add() function should

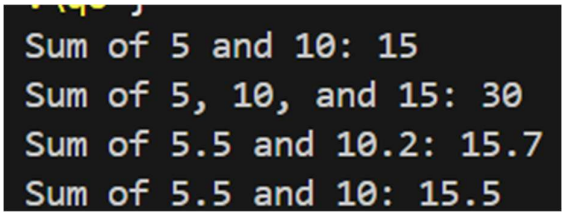
be overloaded to handle the following cases:

- Add two integers
- Add three integers
- Add two floating-point numbers
- Add a double and an integer

```
#include <bits/stdc++.h>
using namespace std;

class Calculator {
public:

    int add(int a, int b) {
        return a + b;
    }


    int add(int a, int b, int c) {
        return a + b + c;
```

```
    }

    float add(float a, float b) {
        return a + b;
    }

    double add(double a, int b) {
        return a + b;
    }
};

int main() {
    Calculator calc;

    cout << "Sum of 5 and 10: " << calc.add(5, 10) << endl;

    cout << "Sum of 5, 10, and 15: " << calc.add(5, 10, 15) << endl;

    cout << "Sum of 5.5 and 10.2: " << calc.add(5.5f, 10.2f) << endl;

    cout << "Sum of 5.5 and 10: " << calc.add(5.5, 10) << endl;

    return 0;
```

}



7. Design a class GradeCalculator with overloaded functions calculateGrade() that return the

grade of a student based on:

● Percentage marks (float)

● Marks out of 100 (int)

● Marks in 5 subjects (int, int, int, int, int)

Use simple grading logic:

A: 90–100, B: 80–89, C: 70–79, D: 60–69, F: <60

```
#include <bits/stdc++.h>

using namespace std;


class GradeCalculator {

public:

  char calculateGrade(float percentage) {

    if (percentage >= 90) return 'A';

    else if (percentage >= 80) return 'B';

    else if (percentage >= 70) return 'C';

    else if (percentage >= 60) return 'D';

    else return 'F';

  }
```

```
    char calculateGrade(int marksOutOf100) {
        return calculateGrade(static_cast<float>(marksOutOf100));
    }

    char calculateGrade(int sub1, int sub2, int sub3, int sub4, int sub5) {
        float percentage = (sub1 + sub2 + sub3 + sub4 + sub5) / 5.0;
        return calculateGrade(percentage);
    }
};
int main() {
    GradeCalculator gc;

    cout << "Grade based on percentage 85.5: " << gc.calculateGrade(85.5f) << endl;
    cout << "Grade based on marks out of 100 (75): " << gc.calculateGrade(75) << endl;
    cout << "Grade based on marks in 5 subjects (80, 90, 70, 60, 50): "
         << gc.calculateGrade(80, 90, 70, 60, 50) << endl;

    return 0;
}
```

```
Grade based on percentage 85.5: B
Grade based on marks out of 100 (75): C
Grade based on marks in 5 subjects (80, 90, 70, 60, 50): C
```

8. Create a class Distance Converter that uses function overloading to convert distances:

- From kilometers to meters: convert(float kilometers)
- From meters to centimeters: convert(int meters)
- From miles to kilometers: convert(double miles, char unit)

Each overloaded function should return the converted distance.

```
#include <bits/stdc++.h>
using namespace std;

class DistanceConverter {
public:
  float convert(float kilometers) {
    return kilometers * 1000;
  }

  int convert(int meters) {
    return meters * 100;
  }

  double convert(double miles, char unit) {
    if (unit == 'k') {
      return miles * 1.6;
    } else {
      return miles * 1609.34;
    }
  }
};

int main() {
  DistanceConverter converter;

  float kilometers = 10.5f;
  cout << "Distance in kilometers: " << kilometers << endl;
```

```
    cout << "Distance in meters: " << converter.convert(kilometers) << endl;

    int meters = 1500;
    cout << "Distance in meters: " << meters << endl;
    cout << "Distance in centimeters: " << converter.convert(meters) << endl;

    double miles = 2.5;
    char unit = 'k';
    cout << "Distance in miles: " << miles << endl;
    cout << "Distance in kilometers: " << converter.convert(miles, unit) << endl;

    return 0;
}
```

ssss

```
Distance in kilometers: 10.5
Distance in meters: 10500
Distance in meters: 1500
Distance in centimeters: 150000
Distance in miles: 2.5
Distance in kilometers: 4
```

9. Develop a simple smart home system that allows a user to manage one or more smart devices

(e.g., Light).

a. Task:

i. Design a Smart Device Class: Implement a C++ class to represent a smart

device.

ii. Define Member Variables and Functions:

1. The class should have member variables to store the device's state

(e.g., status, brightness).

2. Implement member functions to manipulate and access the device's state.

b. System Requirements: Your developed system must fulfil the following requirements,

i. Implement Function Overloading: Use function overloading for one or more actions to provide different ways of controlling the device.

ii. Display Device Status:

1. Your program must display the initial state of the device.

2. After applying various settings and actions, the program should display the updated state.

c. For example: Implement a SmartLight device with the following properties,

i. Name: A string to identify the light (e.g., "Living Room Light").

ii. Status: A boolean or enum to represent its on/off state.

iii. Brightness: An integer value ranging from 0 to 100, where 0 means the light is off.

iv. Functions: togglePower(), changeSetting(int brightness), displayState()

d. Extend Functionality: After successfully implementing the SmartLight, implement the same requirements for a second smart device of your choice.

```
#include <iostream>
#include <string>
using namespace std;

class SmartLight {
   string name;
   bool status;
   int brightness;
public:
   SmartLight(string n, bool s = false, int b = 0){
```

```
    name = n;
    status = s;
    brightness = b;
}

void togglePower() {
    status = !status;
    if (!status) brightness = 0;
}

    void changeSetting(int b) {
    if (b < 0) b = 0;
    if (b > 100) b = 100;
    brightness = b;
    status = (b > 0);
}
void changeSetting(bool s) {
    status = s;
    if (!s) brightness = 0;
    else if (brightness == 0) brightness = 50;
}

void displayState() {
    cout << "Light: " << name << endl;
    cout << "Status: " << (status ? "ON" : "OFF") << endl;
    cout << "Brightness: " << brightness << endl;
    cout << "--------------------------" << endl;
}
```

```
};

class SmartFan {
  string name;
  bool status;
  int speed;
public:
  SmartFan(string n, bool s = false, int sp = 0) {
    name = n;
    status = s;
    speed = sp;
  }

  void togglePower() {
    status = !status;
    if (!status) speed = 0;
  }


  void changeSetting(int sp) {
    if (sp < 0) sp = 0;
    if (sp > 5) sp = 5;
    speed = sp;
    status = (sp > 0);
  }
  void changeSetting(bool s) {
    status = s;
    if (!s) speed = 0;
```

```
        else if (speed == 0) speed = 3;
    }

    void displayState() {
        cout << "Fan: " << name << endl;
        cout << "Status: " << (status ? "ON" : "OFF") << endl;
        cout << "Speed: " << speed << endl;
        cout << "--------------------------" << endl;
    }
};

int main() {
    SmartLight light("Living Room Light");
    SmartFan fan("Bedroom Fan");

    cout << "Initial States:\n";
    light.displayState();
    fan.displayState();



    light.changeSetting(75);
    fan.changeSetting(4);

    light.togglePower();
    fan.changeSetting(false);

    light.changeSetting(true);
    fan.changeSetting(true);
```

```
    cout << "\nUpdated States:\n";
    light.displayState();
    fan.displayState();

    return 0;
}
```

```
Initial States:
Light: Living Room Light
Status: OFF
Brightness: 0
--------------------------
Fan: Bedroom Fan
Status: OFF
Speed: 0
--------------------------

Updated States:
Light: Living Room Light
Status: ON
Brightness: 50
--------------------------
Fan: Bedroom Fan
Status: ON
Speed: 3
--------------------------
```